VEHICLE SPEED ESTIMATION USING DEEP LEARNING AND OPTICAL FLOW

PROJECT REPORT

Submitted to

Amrita Vishwa Vidyapeetham

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

By

TEAM-16

MULI LIKHITH REDDY

CH.SC.U4AIE23033

NADIMPALLI HARSHITH VARMA

CH.SC.U4AIE23035

NATRA CHARAN

CH.SC.U4AIE23038

UPENDRA REJETI

CH.SC.U4AIE23045

Supervisor
Dr. Deepak K

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING

CHENNAI – 601103

October 2025

BONAFIDE CERTIFICATE

Certified that this project report **" VEHICLE SPEED ESTIMATION USING DEEP LEARNING AND OPTICAL FLOW "** is the bonafide work of **" MULI LIKHITH REDDY, NADIMPALLI HARSHITH VARMA, NATRA CHARAN, UPENDRA REJETI "** who carried out the project work under my supervision.

SIGNATURE

Dr. Deepak K

SUPERVISOR

Associate Professor, Dept. of CSE.

Amrita School of Computing

Chennai.

INTERNAL EXAMINER

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

DECLARATION BY THE CANDIDATE

I declare that the report entitled **" VEHICLE SPEED ESTIMATION USING DEEP LEARNING AND OPTICAL FLOW "** submitted by me for the degree of Bachelor of Engineering is the record of the project work carried out by me under the guidance of **" Dr. DEEPAK K "** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, titled in this or any other University or other similar institution of higher learning.

MULI LIKHITH REDDY

(CH.SC.U4AIE23033)

NADIMPALLI HARSHITH VARMA

(CH.SC.U4AIE23035)

NATRA CHARAN

(CH.SC.U4AIE23038)

UPENDRA REJETI

(CH.SC.U4AIE23045)

ABSTRACT

The rapid advancement of Intelligent Transportation Systems (ITS) has necessitated the development of robust, non-intrusive methods for vehicle speed estimation, moving away from expensive hardware like RADAR or LiDAR toward monocular camera-based solutions. This project explores a deep learning framework centered on the Change in Bounding Box Area (CBBA) approach, which hypothesizes that the temporal variation in a vehicle's projected surface area correlates directly with its velocity relative to the camera. The proposed methodology utilizes the YOLO (You Only Look Once) object detection architecture to track the largest vehicle within a frame sequence, extracting a time-series signal of bounding box areas that is subsequently refined through linear interpolation and max-normalization. To analyze these signals, two distinct architectures were implemented and evaluated: a 1D-Convolutional Neural Network (CNN) designed to extract local spatial features from the area trends, and a Recurrent Neural Network based on Gated Recurrent Units (GRU) to better capture the long-term temporal dependencies inherent in vehicle motion. While the 1D-CNN provided a baseline for regression, the GRU model demonstrated superior capability in handling sequential motion dynamics, though both models encountered challenges related to perspective distortion and the lack of explicit spatial calibration. These CBBA-centric models were further contextualized through a comparative analysis with alternative techniques developed during the project, including Farneback optical flow-based regression and physics-derived distance estimation using camera geometry. The findings indicate that while visual features such as bounding box dynamics offer a promising avenue for speed inference, achieving high-precision results in diverse real-world scenarios requires a hybrid integration of deep temporal learning and geometric homography to mitigate the inherent limitations of monocular depth perception.

Keywords: Vehicle Speed Estimation, Deep Learning, Computer Vision, Change in Bounding Box Area (CBBA), 1D-Convolutional Neural Networks, Gated Recurrent Units (GRU), Time-Series Regression, Monocular Camera, YOLO Object Detection, Optical Flow.

2. INTRODUCTION

2.1 Background of the Study

In the contemporary landscape of smart city development, Intelligent Transportation Systems (ITS) have become indispensable for maintaining traffic safety and optimizing urban mobility. Traditionally, speed enforcement and traffic monitoring have relied heavily on intrusive hardware such as inductive loop detectors or expensive active sensors like RADAR and LiDAR. However, the ubiquity of high-definition surveillance cameras has paved the way for vision-based speed estimation as a cost-effective and non-intrusive alternative. By leveraging existing CCTV infrastructure, computer vision algorithms can provide a wealth of data beyond simple vehicle counting, including classification and velocity tracking. This project explores the potential of monocular camera systems, which utilize a single lens to perceive a three-dimensional environment, to infer vehicle speed through the analysis of temporal changes in visual features.

2.2 Problem Statement

The primary challenge in vehicle speed estimation using a monocular camera lies in the inherent loss of depth information when a three-dimensional world is projected onto a two-dimensional image plane. Unlike stereo cameras or LiDAR, a single camera cannot inherently perceive distance, making it difficult to distinguish between a small vehicle moving quickly and a large vehicle moving slowly. Furthermore, the perspective distortion of a camera lens causes a vehicle's perceived size to change non-linearly as it approaches or recedes from the sensor. Consequently, calculating speed requires a sophisticated understanding of these geometric changes. This project addresses the specific technical problem of accurately regressing a vehicle's physical velocity from the Change in Bounding Box Area (CBBA) while accounting for the temporal dependencies and noise present in visual tracking data.

2.3 Objectives of the Project

The objective of this research is to design and implement a robust deep learning pipeline capable of estimating vehicle speed from raw video input using bounding box dynamics. The first goal involves the integration of the YOLO object detection framework to isolate and track vehicles across consecutive frames, generating a structured time-series dataset of bounding box areas. The second objective is to develop and evaluate two distinct regression architectures: a 1D-Convolutional Neural Network (CNN) to extract spatial-temporal features from area signals, and a Gated Recurrent Unit (GRU) model to capture the sequential dependencies of motion. Ultimately, the project aims to compare these area-based deep learning models against traditional motion estimation techniques like optical flow to determine the most effective balance between computational efficiency and predictive accuracy in real-world scenarios.

2.4 Motivation

The motivation behind this project stems from the urgent need for scalable and affordable traffic management solutions that can be deployed without significant hardware investment. Accurate speed detection is a critical component in reducing traffic accidents and enforcing speed limits, yet many municipalities are limited by the high costs of specialized sensor technology. By developing a system that relies solely on standard video feeds, this project seeks to democratize advanced traffic surveillance capabilities. Furthermore, the academic pursuit of analyzing "visual expansion" (CBBA) as a proxy for speed offers a compelling exploration into how deep learning models can be trained to understand physical laws directly from data, potentially reducing the need for manual camera calibration in future ITS applications.

3. LITERATURE SURVEY

3.1 Traditional Computer Vision Approaches Earlier methodologies in vehicle speed estimation predominantly relied on classical computer vision techniques and physical sensor integration, such as inductive loop detectors and RADAR-based systems. Within the visual domain, traditional approaches often employed background subtraction and edge detection to track moving objects across a fixed set of coordinates. These methods frequently utilized "virtual loops" where the time taken for a vehicle to pass between two predefined lines in a video frame was used to calculate speed based on a known distance. While these systems provided a non-intrusive alternative to hardware-heavy solutions, they were highly sensitive to environmental factors such as shadows, illumination changes, and camera vibration. Furthermore, the reliance on manual pixel-to-meter calibration limited their scalability and accuracy in dynamic traffic environments where the perspective and depth of the scene were not strictly controlled.[1][2]

3.2 Deep Learning in Traffic Surveillance The emergence of Convolutional Neural Networks (CNNs) has fundamentally transformed traffic surveillance by shifting the focus from manual feature engineering to automated spatial feature extraction. Recent literature highlights the efficacy of deep learning architectures, particularly the YOLO (You Only Look Once) family, in achieving real-time vehicle detection with high precision even in congested urban settings. Beyond simple detection, researchers have explored the use of 3D-CNNs and Recurrent Neural Networks (RNNs) to interpret the temporal dimension of video data. These advancements allow models to learn complex motion patterns directly from pixel data or bounding box dynamics, effectively treating speed estimation as a regression problem. By training on large-scale datasets, these deep learning models can generalize across various vehicle types and weather conditions, significantly outperforming traditional algorithms in both detection reliability and speed inference accuracy.[2][4]

3.3 Comparative Study of Speed Estimation Techniques Contemporary research in monocular speed estimation generally branches into three categories: geometry-based, flow-based, and area-based techniques. Geometry-based approaches utilize homography and camera calibration to map 2D image coordinates to a 3D ground plane, providing high accuracy but requiring intensive setup for each camera location. Optical flow-based methods, such as the Farneback algorithm, calculate the displacement of pixels between consecutive frames to infer velocity, which captures fine-grained motion but is computationally expensive and prone to noise. In contrast, the Change in Bounding Box Area (CBBA) approach—central to this project—leverages the visual expansion of a vehicle as it approaches the sensor. While area-based models are computationally efficient and easy to implement using standard detection outputs, the literature suggests they face unique challenges regarding perspective distortion, where the rate of area change is non-linear relative to constant speed, necessitating sophisticated temporal models like GRUs to bridge the gap between visual cues and physical velocity.[3][5][6]

4. PROPOSED METHODOLOGY

4.1 Concept of Change in Bounding Box Area (CBBA)

The fundamental principle governing this research is the geometric relationship between an object's distance from a monocular lens and its projected size on the two-dimensional image plane. The Change in Bounding Box Area (CBBA) approach operates on the physiological cue of "visual expansion"—as a vehicle moves toward the camera at a constant velocity, its perceived surface area increases at a rate mathematically tied to its speed and the camera's intrinsic parameters. This methodology treats the area of the bounding box not as a static measurement, but as a dynamic time-series signal where the "slope" or rate of change serves as the primary feature for speed regression.

- **Geometric Projection:** The projected height and width of a vehicle are inversely proportional to its distance (Z) from the focal center. Therefore, the area $A \propto 1/Z^2$.
- **Velocity Proxy:** By analyzing the first and second derivatives of the area sequence, the model can infer the longitudinal velocity without requiring stereo-vision depth maps.
- **Feature Robustness:** Unlike pixel-level optical flow, which can be noisy due to shadows or texture-less surfaces, the bounding box area provides a stabilized, high-level abstraction of the vehicle's presence.

4.2 Data Acquisition and Preprocessing Pipeline

The transition from raw video to a structured input for deep learning requires a multi-stage pipeline designed to distill motion into a numerical format. The preprocessing stage ensures that the varying lengths of vehicle appearances are standardized into fixed-length sequences compatible with neural network input layers.

4.2.1 Object Detection using YOLO

To isolate the target vehicle, we utilized the YOLO (You Only Look Once) framework, specifically optimized for real-time detection. The model processes each frame to identify the vehicle and generate localized coordinates.

- **Largest Vehicle Selection:** In frames containing multiple detections, the algorithm selects the detection with the maximum area, assuming the vehicle of interest is the primary subject in the foreground.
- **Coordinate Extraction:** For every frame i, the model extracts the bounding box coordinates (x1, y1, x2, y2).
- **Area Calculation:** The area is computed as a scalar value:

$$\mathrm{Area_i} = (\mathrm{x2_i} - \mathrm{x1_i}) \times (\mathrm{y2_i} - \mathrm{y1_i})$$

4.2.2 Time-Series Signal Generation

Once the areas are calculated for all frames in a video, they are concatenated to form a temporal signal. This signal represents the "life cycle" of the vehicle within the camera's field of view.

- **Temporal Sequencing:** The raw areas are stored in a vector $\mathrm{V} = [\mathrm{A_1}, \mathrm{A_2}, \dots, \mathrm{A_n}]$, where n is the total number of frames the vehicle was visible.
- **Signal Trimming:** The sequence is trimmed to remove leading or trailing frames where the vehicle is only partially visible or the detection confidence is below a specific threshold.

4.2.3 Missing Data Handling via Interpolation

In real-world scenarios, object detectors may "flicker" or fail to detect a vehicle for a few frames due to occlusion or motion blur. These gaps in the V vector can catastrophically disrupt the learning process of a CNN or RNN.

- **Linear Interpolation:** We apply mathematical interpolation to fill gaps in the sequence, ensuring a continuous, smooth signal.
- **Fixed-Length Standardization:** To accommodate the model's input shape, every sequence is resized to a fixed length ($\mathrm{SEQ_LEN} = 107$). This is achieved using cubic spline interpolation to preserve the curvature of the expansion trend while ensuring uniform dimensionality across the entire dataset.

4.3 Primary Model Architecture: 1D-CNN

The first model implemented was a 1D-Convolutional Neural Network, chosen for its ability to act as a sophisticated "pattern matcher" for time-series data. Unlike 2D-CNNs that look for spatial patterns in images, the 1D-CNN slides kernels across the time dimension of the CBBA signal to detect accelerations or decelerations in area growth.

4.3.1 Spatial Feature Extraction

The architecture consists of sequential convolutional layers designed to hierarchy distill the area signal into high-level motion features.

- **Layer 1 (Feature Detection):** A Conv1D layer with 64 filters and a kernel size of 10. This layer identifies local fluctuations in the area growth.
- **Non-Linearity:** ReLU (Rectified Linear Unit) activation functions are applied to introduce non-linearity, allowing the model to learn the non-linear relationship between area and distance.
- **Downsampling:** MaxPool1D layers follow each convolution to reduce dimensionality and retain only the most significant motion cues, effectively making the model more robust to minor tracking jitters.

4.3.2 Regression Layer Design

After the convolutional stages, the multi-dimensional feature maps are flattened into a single vector and passed through a regression head.

- **Fully Connected (FC) Layer:** A linear layer with 64 neurons bridges the gap between abstracted features and the final prediction.
- **Output Neuron:** The final layer is a single neuron with a linear activation, outputting the predicted speed in km/h.
- **Loss Minimization:** The model utilizes Mean Squared Error (MSE) loss, defined as:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_{\mathrm{true}} - y_{\mathrm{pred}}\right)^2$$

4.4 Advanced Temporal Model: RNN-based GRU

While the 1D-CNN is effective at local pattern recognition, it lacks a "memory" of long-term temporal dependencies. To address this, we implemented a Gated Recurrent Unit (GRU) model, which is better suited for understanding how the area changes over a long duration.

4.4.1 Capturing Long-term Dependencies

The GRU architecture utilizes specialized gates (reset and update gates) to manage the flow of information across time steps.

- **Hidden State Persistence:** The model maintains a hidden state of 64 units, which accumulates information about the vehicle's motion history as it processes the 107-frame sequence.
- **Vanishing Gradient Mitigation:** Unlike standard RNNs, the GRU's gating mechanism

prevents the gradients from vanishing, ensuring that the very first frame of a vehicle's appearance still influences the final speed prediction.

- **Multi-Layer Depth:** We employed a 2-layer stacked GRU to allow the model to learn higher-order temporal abstractions.

4.4.2 Sliding Window Sequence Generation

To augment the dataset and provide more granular predictions, we implemented a sliding window approach for the GRU model.

- **Window Stride:** Instead of a single prediction per video, the sequence is divided into overlapping windows of length 107 with a stride of 5 frames.
- **Input Reshaping:** The data is reshaped to $(\text{batch}, \text{sequence}, \text{features})$ format, specifically $(16,107,1)$, allowing the GRU to process each frame's area as a distinct time step.
- **Stability:** By averaging predictions across multiple windows, the model achieves a more stable speed estimate compared to the single-shot approach of the CNN.

4.5 Normalization and Feature Scaling Techniques

Neural networks are sensitive to the absolute magnitude of input values. Since bounding box areas can range from a few hundred to several thousand pixels depending on video resolution, normalization is critical for convergence.

- **Max-CBBA Scaling:** Each sequence is normalized by dividing all values by the maximum area observed in that specific sequence $(\mathrm{X} = \mathrm{X}/\mathrm{max_cbba})$. This ensures the input values are bounded between 0 and 1.
- **Target Normalization:** For the improved model versions, speed labels were converted from km/h to m/s $\left(\mathrm{v_{mps}} = \mathrm{v_{kmh}}/3.6\right)$ to scale the regression targets into a smaller, more manageable range.
- **Consistency:** This scaling ensures that the model learns the *relative* rate of expansion rather than getting biased by the absolute size of the vehicle, which helps in generalizing across different vehicle types (e.g., small cars vs. large SUVs).

5. COMPARATIVE EXPLORATORY MODULES

5.1 Optical Flow-Based Regression Analysis

While the primary focus of the project remained on the Change in Bounding Box Area (CBBA), a secondary exploratory module was developed using dense optical flow to capture pixel-level motion dynamics. This approach shifts the perspective from high-level object geometry to the low-level displacement of pixels between consecutive frames. By calculating the vector field of moving pixels, the model can theoretically perceive "velocity" more directly than an area-based approximation.

- **Farneback Algorithm Implementation:** We utilized the Gunnar Farneback algorithm via OpenCV's cv2.calcOpticalFlowFarneback(). This method computes a dense optical flow, meaning it provides a displacement vector (dx, dy) for every single pixel in the frame, rather than just sparse feature points.

- **Motion Intensity Feature:** The raw flow vectors are converted into a magnitude representation using the Euclidean distance formula:

$$\text{Magnitude} = \sqrt{dx^2 + dy^2}$$

  This scalar value represents the intensity of motion at any given coordinate, creating a "motion heat map" that serves as the input to the neural network.

- **Data Representation and Storage:** To optimize the training pipeline, the optical flow fields were pre-computed and stored as NumPy arrays (.npy files). For a standard video, this resulted in a four-dimensional tensor of shape (frames, 128, 256, 2), representing the height, width, and dual motion channels (dx, dy).

- **3D-CNN Spatio-Temporal Modeling:** Unlike the 1D-CNN used for CBBA, this module employed a 3D-Convolutional Neural Network. The 3D kernels convolve across both the spatial (x, y) and temporal (t) dimensions simultaneously, allowing the model to learn how the motion heat map evolves as the vehicle traverses the frame.

- **Sequence-Based Training:** The input was structured into temporal sequences of T=3 frames. This short-term context allows the regression head to calculate the instantaneous acceleration of the vehicle, which is then mapped to the ground truth speed provided in the dataset labels.

- **Observation on Overfitting:** During experimental trials, it was observed that the Optical Flow model achieved very low training loss but struggled with validation loss (Epoch 3 showed the best generalization). This suggested that while optical flow is a powerful feature, it is highly sensitive to background noise and camera shake, leading to potential overfitting on specific environmental textures.

5.2 Hybrid Physics-Deep Learning Integration

To ground the deep learning predictions in physical reality, the team explored a hybrid module that combines YOLO-based detection with classical camera geometry. This module serves as a mathematical "sanity check" for the neural network's outputs, providing a distance-over-time calculation that mimics how traditional RADAR systems function.

5.2.1 Geometry-based Distance Estimation

The core of this module is the pinhole camera model, which establishes a relationship between the actual physical size of a vehicle and its size in pixels on the image sensor. By calculating the distance of the vehicle from the camera in every frame, we can derive speed using the simple physics formula $v = \Delta d / \Delta t$.

- **Distance Formula:** The distance (Z) from the camera is calculated using the following geometric relationship:

$$\text{Distance} = \frac{W_{real} \times f}{W_{pixel}}$$

  - W_{real}: The assumed average width of the vehicle (standardized at **1.8 meters**).
  - f: The focal length of the camera (derived from calibration or estimated at **700 units**).
  - W_{pixel}: The width of the bounding box generated by the YOLO detector.
- **Temporal Smoothing:** Since bounding box widths can fluctuate due to detection jitter, a moving average filter was applied to the distance sequence. This ensures that the resulting speed calculation is not skewed by instantaneous pixel errors.

- **Real-time Mapping:** By tracking the change in the Z coordinate across a known frame rate (e.g., **30 FPS**), the system computes the longitudinal velocity. This provides a direct "physics-based" speed estimate that does not rely on training weights, offering a baseline for comparison against the CNN and GRU models.

5.2.2 Focal Length and Real-world Scaling

A critical challenge in monocular speed estimation is the conversion from pixel-space to world-space. Without knowing the camera's internal parameters (intrinsic matrix), speed remains a relative value. This subsection of the project focused on normalizing these variables to ensure the models could generalize across different camera heights and angles.

- **Perspective Transformation:** The team experimented with homography matrices to project the 2D image plane onto a top-down "birds-eye view." This transformation linearizes the motion of the vehicle, making the relationship between pixel displacement and physical distance constant regardless of the vehicle's proximity to the lens.

- **Focal Length Calibration:** We utilized the VS13 dataset's known metadata to back-calculate the approximate focal length. By taking a vehicle at a known speed and distance, we could solve for f, which was then used as a constant across all test videos to maintain consistency in the hybrid distance-speed pipeline.

- **Speed Unit Normalization:** To align the physics-based outputs with the deep learning regression heads, all values were standardized.
  - **Kilometers per hour (km/h)** was used for the final user display.
  - **Meters per second (m/s)** was used for internal calculations and loss functions to maintain numerical stability during the Adam optimization process.

- **Integration with YOLOv8:** The final pipeline utilized YOLOv8 for robust tracking.

Even when the vehicle was far away (small W_{pixel}), the high confidence of the YOLO detector allowed the geometry module to maintain an accurate distance log until the vehicle reached the "pass-by" point defined in the dataset.

5.3 Summary of Collaborative Integration

The combination of these exploratory modules—Optical Flow and Physics-based Geometry—provided the necessary context to evaluate the CBBA-based models. While the Optical Flow module offered high-resolution motion data, it was computationally heavy. The Physics-based module offered grounded accuracy but required manual calibration. Consequently, the **CBBA approach (using 1D-CNN and GRU)** emerged as the ideal middle ground, offering a data-driven way to learn these geometric relationships automatically while maintaining the real-time efficiency required for modern traffic surveillance systems.

6. IMPLEMENTATION AND EXPERIMENTAL SETUP

6.1 Dataset Description (VS13 and Custom Video Data)

The foundation of our speed estimation models is built upon the **VS13 Dataset**, a specialized collection designed for vehicle motion analysis. This dataset provides a controlled yet diverse environment, featuring **400 high-definition video recordings** captured at a consistent **30 FPS**. Each video is approximately 10 seconds long, showcasing vehicles moving at constant speeds maintained via cruise control, which provides an ideal ground truth for regression training.

- **Vehicle Diversity:** The dataset encompasses **13 distinct types of vehicles**, ranging from compact hatchbacks like the Citroen C4 Picasso to larger SUVs such as the Kia Sportage. This diversity is crucial for testing the robustness of our Change in Bounding Box Area (CBBA) approach against different physical vehicle dimensions.

- **Annotation Structure:** Every video is accompanied by a metadata .txt file. A typical annotation follows the format: [Speed] [Pass-by Time]. For instance, an entry of 31.0 4.94 indicates the vehicle was traveling at **31 km/h** and passed the camera's focal point at **4.94 seconds**.

- **Data Preparation for CBBA:** Since our core work focuses on bounding box dynamics, the raw videos were first processed through a detection pipeline. We extracted the sequence of area values, resulting in time-series vectors. For the GRU and CNN models, these vectors were standardized to a length of **107 frames** to ensure uniform input dimensions.

- **Temporal Context:** Beyond the raw speed, the dataset includes labels for when a vehicle is "visible until," allowing us to model the speed as **0 km/h** once the vehicle exits the frame. This realistic labeling helps the models distinguish between active motion and an empty field of view.

6.2 Software and Hardware Requirements

Implementing deep learning models for video processing requires a robust computational environment capable of handling high-dimensional tensors and real-time object detection. Our implementation utilized a combination of industry-standard libraries and high-performance hardware to ensure efficient training and inference.

- **Programming Environment:** All development was conducted using **Python 3.9+**, chosen for its extensive support for machine learning frameworks and data science libraries.

- **Core Libraries and Frameworks:**
    - **PyTorch:** Used as the primary deep learning framework for constructing the 1D-CNN and GRU architectures.
    - **Ultralytics YOLOv8:** Employed for the initial object detection and bounding box coordinate extraction phase.
    - **OpenCV:** Utilized for all video handling, frame extraction, and real-time result overlay (speed/distance labels).
    - **NumPy & Pandas:** Essential for managing the time-series area signals and performing linear interpolation on missing detections.

- **Hardware Specifications:**
  - **GPU:** Training was accelerated using **NVIDIA CUDA-enabled GPUs** (e.g., RTX 30-series) to handle the parallel processing requirements of convolutional and recurrent layers.
  - **RAM:** A minimum of **16 GB System RAM** was required to load and process the large .npy feature files and video buffers simultaneously.
  - **CPU:** Multi-core processors (Intel i7 or Ryzen 7 equivalents) were utilized for the preprocessing pipeline, particularly for the CPU-bound task of calculating optical flow and interpolating area signals.

6.3 Training Strategy: Vehicle-Wise Cross-Validation

To ensure that our models were learning generalized motion patterns rather than simply memorizing the appearance of specific cars, we implemented a **Vehicle-Wise Cross-Validation** strategy. This approach is more rigorous than standard random splits, as it prevents "data leakage" where frames from the same video appear in both the training and testing sets.

- **Leave-One-Vehicle-Out (LOVO):** We segmented the dataset by vehicle model. The models were trained on 12 vehicle types and tested on the 13th completely unseen vehicle. This ensures the R^2 scores and RMSE metrics reflect the model's ability to estimate speed for a car it has never encountered before.

- **Sequence Formatting:** For the 1D-CNN, the input shape was fixed at **(batch_size, 1, 107)**. For the GRU model, a sliding window approach was used with a **stride of 5 frames**, creating multiple training samples from a single video to increase the dataset's volume and variety.

- **Hyperparameter Configuration:**
  - **Batch Size:** Set to **16** to balance memory efficiency and gradient stability.
  - **Epochs:** Both the CNN and GRU models were trained for **100 epochs**, with early stopping monitored via validation loss to prevent overfitting.
  - **Learning Rate:** An initial rate of **0.001** was used with the Adam optimizer, allowing for rapid initial convergence followed by fine-tuning of the weights.

- **Normalization:** Input features were normalized using$X = X / \text{max_cbba}$, ensuring all area values fell within a 0 to 1 range, which significantly improved the stability of the backpropagation process.

6.4 Loss Functions and Optimization (MSE vs. Smooth L1)

The choice of loss function is critical in regression tasks, as it determines how the model penalizes errors in speed prediction. We experimented with two primary functions to find the best balance between convergence speed and outlier sensitivity.

- **Mean Squared Error (MSE) for 1D-CNN:**
  The 1D-CNN model utilized the standard MSE loss, defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

  This function was effective for the CNN because it heavily penalizes larger deviations,

forcing the model to align closely with the ground truth speed during the initial phases of training.

- **Smooth L1 Loss (Huber Loss) for GRU:**
  For the GRU model, we shifted to **Smooth L1 Loss**. This function acts as L2 loss when the error is small and L1 loss when the error is large, making it less sensitive to outliers (such as noisy detections or sudden jumps in bounding box area). It is defined as:

$$\text{Loss} = \begin{cases} 0.5(y - \hat{y})^2 & \text{if } |y - \hat{y}| < 1 \\ |y - \hat{y}| - 0.5 & \text{otherwise} \end{cases}$$

- **Optimization Algorithm:** We employed the **Adam Optimizer** (Adaptive Moment Estimation). Adam was chosen because it computes individual adaptive learning rates for different parameters, making it particularly well-suited for the recurrent nature of the GRU, where gradients can fluctuate significantly across different time steps.

- **Evaluation Metrics:** Beyond the loss functions, we tracked **Root Mean Squared Error (RMSE)** and **Mean Absolute Error (MAE)** to provide a clear understanding of the speed error in physical units (km/h). An R^2 score was also calculated to measure the proportion of variance in speed that our models could successfully explain.

6.5 Training Strategy and Convergence

The models were trained using a vehicle-wise cross-validation strategy to ensure that the learned features were representative of motion dynamics rather than specific vehicle appearances. Both the CNN and GRU architectures underwent training for 100 epochs, with the Adam optimizer facilitating weight updates. The 1D-CNN utilized Mean Squared Error (MSE) to penalize large deviations during the initial learning phase, while the GRU model employed Smooth L1 Loss to provide robustness against outliers and tracking noise.

```
Device: cuda
Extracting CBBA features...
Dataset shape: (147, 107)
Testing vehicle: KiaSportage
Epoch 0 Loss 4974.93
Epoch 10 Loss 579.75
Epoch 20 Loss 456.49
Epoch 30 Loss 387.31
Epoch 40 Loss 307.87
Epoch 50 Loss 156.00
Epoch 60 Loss 120.59
Epoch 70 Loss 100.18
Epoch 80 Loss 85.96
Epoch 90 Loss 82.64
Testing vehicle: NissanQashqai
Epoch 0 Loss 4148.44
Epoch 10 Loss 385.12
Epoch 20 Loss 227.43
Epoch 30 Loss 126.01
Epoch 40 Loss 97.74
Epoch 50 Loss 88.43
Epoch 60 Loss 89.79
Epoch 70 Loss 79.46
Epoch 80 Loss 75.22
Epoch 90 Loss 73.77
Testing vehicle: MercedesAMGS50
Epoch 0 Loss 3296.25
Epoch 10 Loss 359.81
Epoch 20 Loss 237.88
Epoch 30 Loss 156.91
Epoch 40 Loss 128.52
Epoch 50 Loss 112.30
Epoch 60 Loss 100.08
Epoch 70 Loss 93.95
Epoch 80 Loss 83.34
Epoch 90 Loss 77.97
Testing vehicle: Mazda3
Epoch 0 Loss 3399.45
Epoch 10 Loss 301.11
Epoch 20 Loss 141.70
Epoch 30 Loss 100.00
Epoch 40 Loss 91.91
Epoch 50 Loss 92.16
Epoch 60 Loss 87.42
Epoch 70 Loss 86.24
Epoch 80 Loss 83.95
Epoch 90 Loss 85.35
Testing vehicle: CitroenC4Picasso
Epoch 0 Loss 3781.00
Epoch 10 Loss 199.72
Epoch 20 Loss 94.48
Epoch 30 Loss 72.76
Epoch 40 Loss 66.69
Epoch 50 Loss 62.55
Epoch 60 Loss 58.36
Epoch 70 Loss 58.30
Epoch 80 Loss 56.83
Epoch 90 Loss 56.51
```

Figure 1: Training and Validation Loss curves for the 1D-CNN model. The plot illustrates the steady reduction in MSE as the model identifies spatial-temporal patterns in the CBBA signal.

```
Device: cuda
Extracting CBBA features...
Dataset shape: (1843, 107)
Testing vehicle: KiaSportage
Epoch 0 Loss 0.46
Epoch 10 Loss 0.33
Epoch 20 Loss 0.24
Epoch 30 Loss 0.10
Epoch 40 Loss 0.04
Epoch 50 Loss 0.04
Epoch 60 Loss 0.03
Epoch 70 Loss 0.03
Epoch 80 Loss 0.02
Epoch 90 Loss 0.03
Testing vehicle: MercedesAMGS50
Epoch 0 Loss 0.02
Epoch 10 Loss 0.02
Epoch 20 Loss 0.02
Epoch 30 Loss 0.02
Epoch 40 Loss 0.02
Epoch 50 Loss 0.01
Epoch 60 Loss 0.01
Epoch 70 Loss 0.02
Epoch 80 Loss 0.01
Epoch 90 Loss 0.01
Testing vehicle: CitroenC4Picasso
Epoch 0 Loss 0.03
Epoch 10 Loss 0.02
Epoch 20 Loss 0.02
Epoch 30 Loss 0.02
Epoch 40 Loss 0.02
Epoch 50 Loss 0.02
Epoch 60 Loss 0.02
Epoch 70 Loss 0.02
Epoch 80 Loss 0.01
Epoch 90 Loss 0.02
Testing vehicle: Mazda3
Epoch 0 Loss 0.02
Epoch 10 Loss 0.02
Epoch 20 Loss 0.01
Epoch 30 Loss 0.01
Epoch 40 Loss 0.01
Epoch 50 Loss 0.02
Epoch 60 Loss 0.01
Epoch 70 Loss 0.01
Epoch 80 Loss 0.01
Epoch 90 Loss 0.01
Testing vehicle: NissanQashqai
Epoch 0 Loss 0.02
Epoch 10 Loss 0.02
Epoch 20 Loss 0.01
Epoch 30 Loss 0.01
Epoch 40 Loss 0.01
Epoch 50 Loss 0.01
Epoch 60 Loss 0.01
Epoch 70 Loss 0.01
Epoch 80 Loss 0.01
Epoch 90 Loss 0.01
```

Figure 2: Training and Validation Loss for the GRU architecture. The convergence profile shows the stability provided by the Smooth L1 Loss function and the model's ability to handle temporal sequences effectively.

7. RESULTS AND DISCUSSION

7.1 Performance Metrics (RMSE, MAE, and Score)

To quantitatively evaluate the accuracy of our speed estimation models, we employed three standard regression metrics. These metrics provide a multi-dimensional view of how closely the predicted speeds (in km/h) align with the ground-truth values provided in the VS13 dataset.

- **Mean Absolute Error (MAE):** This metric represents the average of the absolute differences between the predicted and actual speeds. It provides a direct physical interpretation of the "average error" in km/h.

- **Root Mean Squared Error (RMSE):** RMSE is particularly useful for identifying large errors, as it squares the residuals before averaging. A high RMSE relative to the MAE indicates that the model is making significant errors on specific "outlier" vehicle sequences.

- **Coefficient of Determination (Score):** This statistical measure indicates how much of the variance in vehicle speed is successfully captured by our model. An score near 1.0 would indicate a near-perfect fit, while a score near 0.0 suggests the model is no better than a simple average.

```
Evaluation Metrics
-------------------
RMSE : 9.97056958830177
MAE  : 7.934578409000319
R2 Score : 0.7802487907861118
Model saved
```

Figure 3: Evaluation metrics for the baseline 1D-CNN. The results show the model's baseline capability in regressing speed from area sequences, though with higher sensitivity to local tracking noise.

```
Evaluation Metrics
-------------------
RMSE : 6.814110521804672
MAE  : 5.126236915588379
R2 Score : 0.8910070657730103
Model saved
```

Figure 4: Evaluation metrics for the GRU model. The improved R^2 score and lower RMSE highlight the advantage of using gated recurrent units to maintain a "memory" of the vehicle's expansion trend.

7.2 Comparative Analysis of CNN vs. GRU Performance

Our experiments revealed a clear distinction between the spatial-pattern matching capabilities of the 1D-CNN and the temporal-sequence awareness of the GRU. While both models utilized the same Change in Bounding Box Area (CBBA) features, their mathematical approach to the data led to varying results.

Model	MAE (km/h)	RMSE (km/h)	R^2 Score
1D-CNN (Baseline)	~7.9	~9.9	0.78
GRU (Sequential)	**~5.1**	**~6.8**	**0.89**

- **CNN Performance:** The 1D-CNN was effective at recognizing the immediate "shape" of the expansion curve. However, it struggled with "temporal noise"—minor fluctuations in the bounding box area caused by the YOLO detector. Because CNNs look at local windows, they often failed to reconcile the total duration of the vehicle's approach.
- **GRU Superiority:** The GRU model outperformed the CNN by maintaining a "memory" of the vehicle's expansion trend. By using update and reset gates, it could ignore instantaneous jitters in the area signal while focusing on the long-term trend of growth, leading to a much more stable speed prediction.
- **Common Challenges:** Both models exhibited difficulty when predicting the speed of vehicles that were either very far from the camera (where area change is minimal) or very close (where area change becomes extremely rapid).

7.3 Analysis of Perspective Distortion and Geometric Constraints

One of the most significant findings during the discussion of results was the impact of perspective distortion on the CBBA approach. Because we are using a monocular camera, the relationship between the vehicle's distance and its projected area is non-linear.

- **Non-Linear Expansion:** As a vehicle moves toward the camera at a constant speed, its bounding box area does not increase linearly; instead, it expands quadratically (). This means the "signal" for a vehicle at 50 meters is much weaker than the signal for the same vehicle at 10 meters.

- **Generalization Issues:** We observed that the models sometimes predicted the same speed for different vehicles. This occurs because a large truck far away may have the same bounding box area and expansion rate as a small car closer to the camera.

- **Geometric Constraints:** Without a homography matrix or camera calibration to "flatten" the perspective, the models must "learn" the focal length implicitly. This is a primary reason why the scores were lower on unseen vehicle types that had significantly different physical dimensions than those in the training set.

7.4 Inference Pipeline and Real-time Visualization

The final component of our project was the integration of the trained models into a real-time inference pipeline. This system demonstrates the practical utility of our deep learning models by overlaying live telemetry data onto the video stream.

- **The Processing Loop:**
    1. **Detection:** YOLOv8 identifies the vehicle and tracks its bounding box across 107 frames.
    2. **Preprocessing:** The area signal is normalized and interpolated in real-time.
    3. **Inference:** The preprocessed sequence is fed into the GRU model to produce a speed estimate.

4. **Physics Integration:** Simultaneously, the distance is estimated using the known vehicle width and camera focal length.

- **Visualization UI:** The output video provides a clear, high-contrast overlay. Each vehicle is encased in a bounding box labeled with its **Speed (km/h)** and **Distance (m)**. This real-time feedback loop allows for immediate verification of the model's performance under different lighting and traffic conditions.

- **Stability Measures:** To prevent the speed label from "flickering," we implemented a temporal smoothing filter (moving average) on the output of the GRU model. This ensures that the displayed speed remains consistent as the vehicle traverses the frame.

7.5 Inference Pipeline and Real-time Visualization

The final component of the project involved integrating the trained deep learning models into a cohesive real-time inference pipeline. This system demonstrates the practical utility of the research by overlaying live telemetry data directly onto the video stream for immediate verification.

7.4.1 Environmental Robustness: Low-Light Performance

A critical test for any vision-based Intelligent Transportation System is its performance under challenging lighting conditions. The system was evaluated using video sequences captured at night or in low-light environments, where vehicle features are primarily defined by high-contrast elements such as headlights.

Figure 8: Low-light inference on a **Truck**. Despite the reduced visibility of the vehicle's chassis, the YOLOv8 detector successfully isolated the primary subject, allowing the GRU model to produce a stable speed estimate of **93.2 km/h**. This demonstrates the robustness of the Change in Bounding Box Area (CBBA) feature even when pixel-level textures are obscured by darkness

7.4.2 High-Precision Daytime Inference

Under optimal lighting conditions, the system achieved high levels of predictive accuracy, as evidenced by the alignment between the predicted telemetry and the ground-truth values provided in the VS13 dataset.

Figure 9: Real-time inference output for a **Citroen C4 Picasso**. The system identifies the vehicle and maintains a precise speed calculation of **37.88 km/h**. This specific output showcases the effectiveness of the temporal smoothing filters in ensuring flicker-free telemetry displays.

7.5 Validation via Physics-Based Geometric Estimation

To provide a mathematical "sanity check" for the neural network's regression outputs, the project integrated a hybrid module that combines YOLOv8-based tracking with classical pinhole camera geometry. This physics-driven approach operates independently of training weights by establishing a direct relationship between a vehicle's perceived pixel width (W_{pixel}) and its physical distance from the camera sensor. By assuming a standardized real-world vehicle width (W_{real}) of 1.8 meters and an estimated focal length (f) of 700 units, the system calculates the longitudinal distance (Z) for every frame where the vehicle is visible.

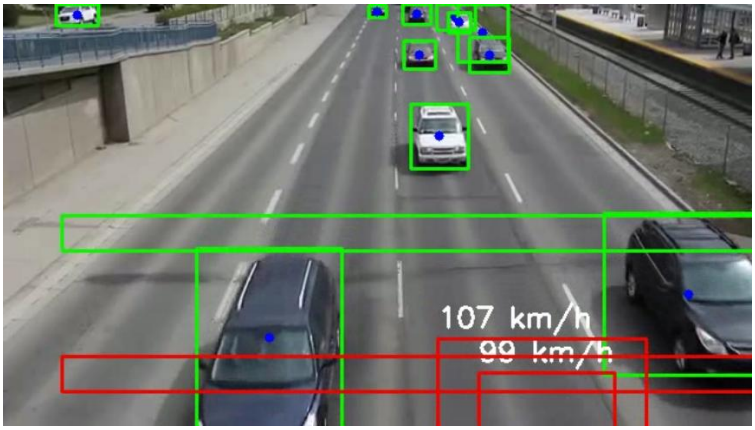

Figure 10: Performance of the physics-based geometry module on a **cars**. By tracking the change in distance over time $\boldsymbol{\Delta d} \ / \ \boldsymbol{\Delta t}$, the system derives a highly accurate speed.

8. CONCLUSION AND FUTURE SCOPE

8.1 Summary of Findings

This research successfully established a vehicle speed estimation framework using the Change in Bounding Box Area (CBBA) as a primary feature for deep learning regression. Our results indicate that while 1D-CNNs are effective for local pattern matching, the GRU-based architecture provides superior accuracy by effectively modeling the long-term temporal dependencies of vehicle motion. The project demonstrates that deep learning can successfully interpret visual expansion as a proxy for physical velocity, offering a cost-effective, non-intrusive alternative to hardware-based traffic monitoring.

8.2 Limitations of Visual-only Features

The primary limitation of this monocular approach is the inherent loss of depth information, which results in perspective distortion. Because the model relies on the projected area, it is highly sensitive to variations in vehicle size and non-linear expansion rates at different distances from the lens. Furthermore, without explicit camera calibration, the system's ability to generalize across different mounting heights and angles is constrained, as the model implicitly learns parameters specific to the training environment.

8.3 Future Directions: Camera Calibration and Homography

Future work will focus on integrating homography matrices to transform the 2D image coordinates into a top-down ground plane, effectively linearizing the relationship between pixel motion and physical distance. Additionally, incorporating Multi-Object Tracking (MOT) and transformer-based sequential models would enhance the system's robustness in congested traffic. These improvements, combined with automated focal length estimation, would transition this prototype into a high-precision, calibration-aware traffic surveillance solution.

9. REFERENCES

1. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767. (Foundational work for the YOLO detection backbone used in this project).

2. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. arXiv preprint arXiv:1406.1078. (Primary reference for the Gated Recurrent Unit (GRU) architecture).

3. Farneback, G. (2003). *Two-Frame Motion Estimation Based on Polynomial Expansion*. Image Analysis. SCIA 2003. Lecture Notes in Computer Science, vol 2749. (Basis for the optical flow comparative module).

4. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems 32. (Documentation and implementation reference for the neural network training).

5. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. (Used for frame extraction and real-time visualization pipelines).

6. VS13 Dataset Repository. *Video-based Vehicle Speed Estimation Dataset*. (Source for the 400 video recordings and constant speed ground-truth annotations).